Microsoft Foundry: from prototype to operated service

How to use this script

Read the SAY and TRANSITION blocks aloud. Everything in DO, COMMAND, EXPECT and CHECK is a stage direction, not something to read to the audience. Conditional SAY blocks replace the normal wording; do not read every branch. Read success wording only after the EXPECT check is satisfied. If it is not, use the relevant conditional wording and the recorded fallback instead. Waiting commentary is optional: skip it when the result is already available. Use the HTML or Markdown version when copying commands. The PDF is for reading and printing; its visual line wraps can introduce unwanted breaks when copied.

The clock starts when you begin slide 1. Slide 15 starts at minute 50. The five hidden appendix slides are answers to questions, not additional presentation material. The script assumes a mixed business/technical customer audience.

The recorded evidence is from 8 November 2026: baseline version 2 passed seven of nine synthetic cases; corrected version 4 passed nine. These are recorded facts, not a promise that a fresh model response will be identical. No live Azure calls or deployments are required merely to read this script.

Before the audience arrives

DO

Open PowerPoint in Presenter View. Keep these windows ready, in this order: 1. A PowerShell terminal in the demo directory. 2. VS Code with instructions.md, volt/domain.py, core.jsonl and regression.jsonl. 3. The recorded evidence HTML, already loaded. 4. The Foundry agent, Traces, and completed native evaluation. 5. GitHub Actions with the blocked, passing and live-release runs.

Close unrelated tabs and notifications. Use large terminal/editor text. Do not display .env files, azd env get-values, or an unfiltered azd ai agent show response; those can expose configuration values.

COMMAND - RUN BEFORE PRESENTING

```
Set-Location C:\Users\sarrabelly\Documents\GitHub\foundry-showcase\demo
$versions = Get-Content evidence\rehearsal-index.json -Raw | ConvertFrom-Json
$baseline = [string]$versions.baseline_version
$improved = [string]$versions.improved_version
$release = Get-Content evidence\release.json -Raw | ConvertFrom-Json
if ([string]$release.version -ne $improved) { throw "Release pointer and rehearsal version
differ. Rehearse before presenting." }
$sessionDir = Join-Path (Get-Location) ("evidence\presenter-" + (Get-Date -Format 'yyyyMMdd-
HHmmss'))
New-Item -ItemType Directory -Path $sessionDir | Out-Null
$baselineReport = Get-Content evidence\baseline.json -Raw | ConvertFrom-Json
$candidateReport = Get-Content evidence\candidate.json -Raw | ConvertFrom-Json
$nativeEvaluation = Get-Content evidence\foundry-evaluation.json -Raw | ConvertFrom-Json
$referenceBaseline = Get-Content evidence\missing-batch-baseline.json -Raw | ConvertFrom-Json
$referenceImproved = Get-Content evidence\missing-batch-improved.json -Raw | ConvertFrom-Json
Start-Process (Resolve-Path 'presentation\Recorded demo evidence.html')
```

```
Start-Process $nativeEvaluation.report_url
Start-Process $versions.intentional_blocked_workflow
Start-Process $versions.passing_quality_workflow
Start-Process $versions.release_workflow
```

CHECK

The retained versions should be 2 and 4 for this package. The completed reports should show 7/9 and 9/9. If the resources or version IDs have changed, reconcile them before presenting; do not substitute a guessed version.

For the live path, warm the exact versions before the room opens. These commands invoke the real model and incur usage. They write new files, leaving the recorded fallback intact. Skip them if you have deliberately chosen the recorded path.

COMMAND - LIVE-PATH WARM-UP, NOT ON STAGE

```
python scripts\demo.py ask "Check Volt Charge 65 batch CH-0926 for a recall." --version
$improved --out "$sessionDir\warm-improved.json"
python scripts\demo.py ask "I have a Volt Charge 65. Is it safe?" --version $baseline --out
"$sessionDir\warm-baseline.json"
```

CHECK

Confirm both complete successfully. In Foundry, use the project and agent bookmarks from docs\DELIVERY.md. Pre-open one version-4 trace with its tool spans. For recorded traces, select the last seven days or a custom window including 8 November 2026; an increasingly narrow "last 24 hours" window can hide them. If the portal's navigation differs, find the page now, not during the session.

If authentication, networking or the model is unavailable, choose the recorded path before starting. Say so at the first demonstration. Do not attempt IAM, package installation, deployment or portal troubleshooting in front of the room.

Running order

Clock	Slides / surface	What to deliver
00:00-02:00	1-2	Welcome and operating story
02:00-04:00	3	Volt & Co and its checkable business contract
04:00-07:00	4	Inner loop, outer loop and evidence bridges
07:00-09:00	5	GitHub / Copilot SDK / Foundry responsibility boundary

09:00-16:00	6 + live surfaces	Shopping, recall and execution evidence
16:00-18:00	7-8	Interpret the scorecard and the three types of evaluation
18:00-25:00	9 + live surfaces	Baseline defect, correction and comparison
25:00-27:00	10	Release policy and the reference outer loop
27:00-34:00	11 + live surfaces	Real GitHub checks, release evidence and pinned invocation
34:00-35:00	12	Why operations must feed development
35:00-47:00	13 + live surfaces	Observation, review, new test and focused recheck
47:00-49:00	14	Four takeaways and customer pilot framing
49:00-50:00	Buffer	Absorb delay; do not add another topic
50:00-60:00	15; appendix only if needed	Q&A

Slide 01 - Microsoft Foundry: from prototype to operated service

TIME

00:00-00:45. Welcome. Advance to slide 2 by 00:45.

DO

Leave the cover visible. Look at the audience rather than the laptop. Start the presentation clock only after completing the pre-show setup. It is at the front of this script and at the end of this slide's PowerPoint note, outside the timed hour.

SAY

Hello, everyone. I'm Sridhar. Today I want to show you Microsoft Foundry through one complete story, rather than take you through a catalogue of features.

We will build with GitHub Copilot, run an agent using the Copilot SDK in Foundry, and examine what it takes to evaluate, release and improve that agent.

The question I would like you to keep in mind is this: when an assistant gives a convincing answer, what evidence would you need before putting it in front of your customers or colleagues?

We will spend about fifty minutes on that journey and keep the final ten for your questions.

TRANSITION - SAY

Let me start with the difference between getting an answer and operating a service.

DO

Advance to slide 2.

Slide 02 - One customer request. A complete operating story.

TIME

00:45-02:00. Do not explain product features individually here.

DO

Point once from Build to Improve. Pause briefly on Prove and Release.

SAY

Most demonstrations begin with a question and end with an impressive response. That is useful, but it leaves several important questions unanswered.

Where did the facts come from? Did the agent use the right tool? If we change its instructions tomorrow, how do we know we have not broken something that worked today? And when a user asks a question we did not anticipate, how does that observation become a better next version?

Those are the connections we will follow. First we build something useful. Then we connect it to facts we can check. We prove specific behaviours, make an explicit release decision, and use operational evidence to improve the next candidate.

That is where Foundry becomes more than a place to try a model. It gives us runtime and lifecycle capabilities around the application. We still own the business logic and the policies that decide what is acceptable.

Today, one small customer journey will make those responsibilities visible.

TRANSITION - SAY

Our customer for that journey is a fictional electronics retailer called Volt & Co.

DO

Advance to slide 3.

Slide 03 - Meet Volt & Co

TIME

02:00-04:00. Establish the data and boundaries once; refer back later.

DO

Point to the customer's request, then the three charger rows. Pause for three seconds after the question below; do not open a long discussion.

SAY

Volt & Co wants an assistant that helps customers choose the cheapest product that actually meets their needs.

Our customer has a Volt Book 14 and wants a charger for less than fifty euros. Looking at these three options, which one should the assistant recommend?

DO

Pause for three seconds. If someone chooses the cheapest, acknowledge the price and point to the 65W requirement. Otherwise continue without waiting for a reply.

SAY

The thirty-watt charger is cheapest, but it does not meet the laptop's sixty-five-watt requirement. The one-hundred-watt charger is compatible with the power requirement, but it is over budget. That leaves the sixty-five-watt model at thirty-nine euros ninety.

This is deliberately simple. We can check the answer against the data rather than argue about whether it sounds persuasive.

The same assistant can also check recall information for a particular product batch. That adds an important distinction: not knowing the batch is not the same as knowing that there is no recall.

All names, prices, specifications and recall records in this demonstration are fictional. The tools are read-only. We are not placing orders, taking payments or changing customer records.

That gives us a useful journey with clear boundaries: find a product, verify compatibility, and handle missing information honestly.

TRANSITION - SAY

Now let's place that one journey inside the development and operating lifecycle.

DO

Advance to slide 4.

Slide 04 - One continuous system: inner and outer loop

TIME

04:00-07:00. Allow time for the audience to inspect the diagram.

DO

Trace the left-hand loop, the upper forward bridge, the right-hand loop and finally the lower return bridge. Do not read every box.

SAY

This is the mental model for the rest of the session. There are two loops and, just as importantly, two bridges between them.

The inner loop, on the left, is the developer's learning cycle. We define the behaviour, connect the tools, run the agent, inspect what happened, evaluate it against expectations, and improve the candidate.

The important word is candidate. A developer saying "this looks better" is not yet a release decision.

Look at the upper bridge. What moves forward is not simply a copied prompt. It is a versioned candidate with evidence: which configuration was used, which cases were evaluated, what passed, what failed, and what boundaries were checked. That is what makes a change reviewable.

The outer loop, on the right, is the repeatable release and operating process. It applies the gates, puts an identified version into service, and watches what that version does when it receives requests.

Now follow the lower arrow back to development. This is the connection that is often missing. An operational observation is reviewed, the expected behaviour is defined, and a new test becomes part of what the next candidate must prove.

That human review is important. A response found in a trace is not automatically the right answer, and a model-generated finding is not automatically the right fix.

This picture is the broader reference architecture. We will not demonstrate every capability on it. Our compact implementation uses local development and a hosted demo target. It is not a separately isolated customer production environment.

What we will demonstrate is both evidence bridges: candidate to release, and observed behaviour back to a test. Keep those two arrows in mind. They connect everything that follows.

TRANSITION - SAY

With the lifecycle clear, let's separate the roles of GitHub, the Copilot SDK and Foundry in the implementation.

DO

Advance to slide 5.

Slide 05 - Build in GitHub. Run and improve in Foundry.

TIME

07:00-09:00. Explain the responsibility boundary, not the whole platform catalogue.

DO

Point to GitHub Copilot SDK in the BUILD row, then Hosted agents in the HOST row, then the three lifecycle phrases along the bottom.

SAY

There are two uses of Copilot in this story, and I want to distinguish them.

First, GitHub Copilot helps the developer write and understand the application code, tests and workflow. That is the development experience.

Second, the GitHub Copilot SDK is the harness inside this particular application. It manages the agent interaction with the model and the tools we have exposed. It is not the same thing as handing our customers an unrestricted coding assistant.

In this implementation, the customer-facing runtime has three allowlisted, read-only business tools. It is not given general shell or filesystem access.

Foundry then hosts that application and provides the endpoint and the surrounding lifecycle capabilities. The model also runs through Foundry, authenticated with Microsoft Entra. We have not embedded a GitHub token in the business agent.

The graphic shows that hosted agents are not limited to this one framework. We chose the Copilot SDK because it makes our GitHub-to-Foundry story concrete. The framework choice does not remove our responsibility for the tool logic, permissions, evaluation coverage or release policy.

The managed Foundry Toolbox and API Management are useful extension points, but they are not configured in this sample. Our three tools are packaged with the agent. I will distinguish what we actually run from the broader platform architecture.

TRANSITION - SAY

Let's move from the architecture to the customer request. The infrastructure is already prepared, so we can spend the time on behaviour and evidence.

DO

Advance to slide 6. Keep it visible during the first transition sentence.

Slide 06 - Watch the answer—and the path that produced it

TIME

09:00-10:00 transition; 10:00-12:15 shopping; 12:15-14:15 recall; 14:15-15:45 trace; 15:45-16:00 return to slides.

DO

Switch to the prepared terminal. The preflight variables from the front of this script must already be set. Keep the recorded evidence HTML one click away. Give each live request a 30-second on-stage budget. If it is still pending, use the IF SLOW wording below and the recorded path for the rest of this segment. Do not paste another command into a terminal that is still busy.

SAY

We are now leaving the slide and sending a request to the deployed agent. The data is synthetic, but this request will use the actual Copilot SDK harness, Foundry model and hosted runtime.

I am selecting the corrected version explicitly. That keeps the behaviour tied to a known candidate rather than whatever happens to be the latest version.

COMMAND

```
python scripts\demo.py ask "Find the cheapest charger for my Volt Book 14 under EUR 50." --
version $improved --out "$sessionDir\shopping.json"
```

WHILE WAITING - SAY IF NEEDED

While it works, notice what we have not done. We have not told the model which charger to choose. We have given it a task and access to a small set of business functions. The interesting evidence will be the facts those functions return and how the final recommendation relates to them.

EXPECT

Successful command; decision recommend; product VC-C65; EUR 39.90 in the answer. Tool evidence includes catalog lookup and compatibility for the selected item. The exact wording and number/order of calls can vary. Do not narrate a call that is not present.

DO

Only after a successful command, use the following to show the answer and tool records without scrolling through the full envelope.

COMMAND

```
$shopping = Get-Content "$sessionDir\shopping.json" -Raw | ConvertFrom-Json
$shopping.output | Format-List
$shopping.tool_calls | ConvertTo-Json -Depth 10
```

SAY

The recommendation is the sixty-five-watt charger. What makes that answer useful is not just the product name: it meets the power requirement and the budget.

Here is the tool evidence behind it. The catalog supplies the prices and specifications. The compatibility check supplies the relationship to the device. We can inspect the path instead of treating the final sentence as our only source of confidence.

Now let's stay with the same retailer and ask the support question.

COMMAND

```
python scripts\demo.py ask "Is my Volt Charge 65 from batch CH-0722 safe to use?" --version
$improved --out "$sessionDir\recall.json"
```

EXPECT

Decision recalled; product VC-C65; batch CH-0722; getRecallNotices appears in tool_calls. The response refers the fictional customer to human support. This sample does not create a support ticket or perform a real handoff.

COMMAND

```
$recall = Get-Content "$sessionDir\recall.json" -Raw | ConvertFrom-Json
$recall.output | Format-List
$recall.tool_calls | ConvertTo-Json -Depth 10
```

SAY

This batch is marked as recalled in the fictional register. The response directs the customer to support rather than inventing a repair workaround.

Notice the scope of the answer: a product and a batch, backed by a returned record. A catalog match is not a blanket safety guarantee. That distinction will matter again when the batch information is missing.

DO

Switch to the pre-opened Foundry Traces view for the agent. Select the rehearsed version-4 request if the new trace has not arrived yet. Open the request details and the custom tool span. If using a recorded trace, use the next sentence.

SAY

I am opening a captured rehearsal trace here so we do not wait for telemetry ingestion. The live request and its tool output are in the terminal; this view shows how we investigate an execution afterward.

The trace gives us an operational record: the invocation, the instrumented tool work and the timing. It is not a window into a model's private chain of thought. We use the observable execution to answer practical questions: which operation ran, with what input, and what happened next?

IF SLOW - SAY

The live service is taking longer than the time I have allocated. I will switch to the captured rehearsal result from the same project and identify it as recorded, rather than make the room wait.

DO

Use presentation\Recorded demo evidence.html and the candidate report. Do not reclassify the recorded request as live. If the command is still running, leave it running in its terminal while showing the capture; do not issue another identical request or overwrite the output file.

IF DIFFERENT RESULT - SAY

This request did not produce the expected result. I will not call it a pass. We would inspect this execution and retain it as evidence. For the planned walkthrough, I will use the clearly labelled rehearsal capture.

RETURN TO SLIDES - SAY

We have now seen a useful answer and the evidence behind one execution. That still leaves a bigger question: does the behaviour hold across a set of cases?

DO

Return to PowerPoint and advance to slide 7 by minute 16.

Slide 07 - From a convincing answer to measurable evidence

TIME

16:00-17:00. Keep the interpretation precise.

DO

Point to the two missing cells in the baseline row and then the corrected row. Do not describe the result as a general accuracy percentage.

SAY

Here is the recorded comparison from rehearsal. The baseline passed seven of nine cases; the corrected version passed all nine.

The two failures both involved missing batch information. That gives us a specific engineering question, not simply a disappointing average score.

These are nine synthetic cases that we deliberately chose. They are not a statistically representative measure of every future customer request, and nine out of nine is not a safety certification.

Their value is that the expectations are visible, the same cases are used for the comparison, and we can connect each result to an identified agent version.

We can now ask whether a change improved the behaviour we intended without silently breaking the cases that already worked.

TRANSITION - SAY

To make that evidence useful, we also need the right kind of check for each question.

DO

Advance to slide 8.

Slide 08 - Three checks. Three different jobs.

TIME

17:00-18:00. This is the explanation for the two different reports in the demo.

DO

Point left to right: Code-based, Model-based, Human.

SAY

For objective facts, use code. In our example, we can check the selected product, the required tool evidence and whether missing input receives the correct status.

For semantic qualities, a model judge can add another signal. Our native Foundry scorecard evaluates the coherence of captured live responses. That is useful, but coherence is not the same as correct recall logic.

And for ambiguous expectations, risk decisions and acceptance, a person still has a role. Someone has to decide what the agent should do when evidence is missing and validate that the test labels reflect the business requirement.

These checks complement each other. A fluent response can fail a business contract, and a mechanically correct status can still produce a poor customer experience.

TRANSITION - SAY

Let's look at the particular defect behind those two failed cases and the small correction that addresses it.

DO

Advance to slide 9, then switch to the prepared terminal and editor.

Slide 09 - Change the contract—not just the wording

TIME

18:00-19:30 inspect the failure; 19:30-21:30 inspect the correction; 21:30-23:00 focused live request; 23:00-24:40 native scorecard; 24:40-25:00 return to slides.

SAY

The enquiry is: "I have a Volt Charge 65. Is it safe?"

There is a missing fact: the batch code. In the baseline, the tool incorrectly treated that absence as a negative recall lookup and returned no_recall.

I want to be explicit: this is a deliberately prepared defect in the demo's business logic. I am not claiming we just discovered a new production incident. The point is to make the failure reproducible and show how we correct it.

DO

In the recorded evidence HTML, scroll to the two failed baseline cases and then the feedback moment. Alternatively show the recorded result directly.

COMMAND

```
$referenceBaseline.output | Format-List
$referenceBaseline.tool_calls | ConvertTo-Json -Depth 10
```

SAY

The wording may sound cautious, but look at the structured decision and the tool result. With no batch supplied, no_recall is not a justified batch-specific conclusion. The problem is in the contract, not simply the tone.

That is why inspecting only the final prose can miss an important defect.

DO

Switch to VS Code. Open src\github-copilot-invocations\volt\domain.py. Use Find for "def recall_notices". Show the if-not-batch branch and the two explicit variants. Do not edit the deployed serving version.

SAY

Here is the change. The baseline branch is retained for demonstration. The corrected branch returns needs_batch when that information is absent.

The agent can then ask a useful clarifying question instead of treating unknown information as a negative result.

We did not need to change the model or introduce another agent to fix this particular defect. We needed a better-defined business rule and a test that would catch the wrong behaviour.

DO

Return to the terminal. Run these separately so the audience can see the deliberate BLOCK followed by PASS. Exit code 1 from the baseline is expected. For the later hosted request, use the recorded candidate result if it has not completed within 30 seconds; do not retry during the presentation.

COMMAND

```
python scripts\check_contract.py --variant baseline
python scripts\check_contract.py --variant improved
```

SAY

These two commands test the business function offline. They are fast, deterministic checks; they do not call a model or prove that the hosted agent used the function correctly.

For that, we also invoke the actual hosted candidate.

COMMAND

```
python scripts\demo.py ask "I have a Volt Charge 65. Is it safe?" --version $improved --out
"$sessionDir\missing-batch-improved.json"
```

WHILE WAITING - SAY IF NEEDED

This separates two failure modes. If the function is wrong, fix the function. If the function is correct but the agent uses it incorrectly, investigate the agent execution. Keeping those checks separate makes diagnosis much faster.

EXPECT

Decision needs_batch, with a customer-facing request for the batch code. getRecallNotices must return needs_batch. Other read-only lookups may also appear; do not promise an exact call sequence.

COMMAND

```
$improvedLive = Get-Content "$sessionDir\missing-batch-improved.json" -Raw | ConvertFrom-Json
$improvedLive.output | Format-List
$improvedLive.tool_calls | ConvertTo-Json -Depth 10
```

SAY

Now the response asks for the missing fact. More importantly, the decision and the tool result agree on why that fact is needed.

The completed full-suite comparison also retains the original shopping, compatibility and recall cases. Correcting this case should not be permission to ignore the behaviours that already mattered.

DO

Switch to the already-open native Foundry evaluation report. Use the URL in evidence\foundry-evaluation.json. Open the evaluator/results view, select one captured answer and point to the coherence result. Do not launch a new job.

SAY

This is a native Foundry model-judged scorecard over captured live responses. It is measuring coherence. It is not replaying these requests now, and it is not the business-contract gate we just examined.

That distinction is useful: a response can be coherent while its underlying business decision is wrong. We need the complementary evidence, not one score that we treat as universal truth.

IF SLOW - SAY

I will use the completed comparison rather than start another batch run. It identifies the version, cases and results, so the evidence remains inspectable.

DO

If either the live request or portal is unavailable, use candidate.json and the recorded HTML. Describe the native run as completed during rehearsal; do not invent a score or claim a result that is not visible.

IF DIFFERENT RESULT - SAY

This candidate did not meet the expected behaviour on this request. That is evidence to keep, not a result to explain away. The next step would be to inspect the trace and withhold approval for that change.

RETURN TO SLIDES - SAY

We have a candidate and evidence about its behaviour. Now we need the process that decides whether that candidate is allowed to be used.

DO

Return to PowerPoint and advance to slide 10 by minute 25.

Slide 10 - How a candidate becomes an operated service

TIME

25:00-27:00. Explain the production reference and the demo boundary together.

DO

Point to DEV, Rebuild, Gate, Promote, then the return arrow. Finally point to the blue caption that describes this demo.

SAY

This is the reference outer loop: a candidate leaves development, is rebuilt and checked, and only then is promoted for use.

The purpose of rebuilding is reproducibility. The purpose of the gate is to make the acceptance criteria executable. The purpose of promotion is to make the serving version identifiable and controlled.

None of that happens merely because an evaluation page exists. We have to configure the workflow and decide which conditions stop it.

For a production implementation, that includes the appropriate environment separation, deployment identities, approvals and rollback arrangements. Those are engineering and governance decisions, not decorative boxes on a slide.

Our compact demo is narrower. Development is local, and the hosted target is a demo project. Our release record pins the version that the demonstration client will invoke. It does not change the default routing in the Foundry portal.

That distinction lets us demonstrate the evidence gate without pretending this is a complete production-isolation reference deployment.

Finally, look at the return arrow. Release is not the end of the lifecycle. We still need to learn from the behaviour of the version we put into service.

TRANSITION - SAY

Let me show you the actual workflow and the record that connects a release decision to a specific hosted version.

DO

Advance to slide 11 and move to the prepared GitHub Actions tabs.

Slide 11 - A release we can explain

TIME

27:00-28:00 transition; 28:00-30:00 contract checks; 30:00-31:30 live workflow; 31:30-33:30 release record and request; 33:30-34:00 return.

SAY

I have completed the deployment and the longer evaluation run during rehearsal. I will show their actual records rather than spend our session waiting for a build.

First, this is the deliberately blocked business-contract run.

DO

Select the tab opened from $versions.intentional_blocked_workflow. Open the tool-contract job, then "Offline business contract gate". Point to the failing step. This run deliberately selects baseline. Do not select the older OIDC setup failure.

SAY

The baseline fails the missing-batch contract, so this check does not return success. That failure is intentional and meaningful.

This particular workflow is credential-free and offline. It tests the code contract. It has not invoked an LLM, and I do not want to imply that it has.

DO

Select $versions.passing_quality_workflow. Show the successful corresponding contract step. Then select $versions.release_workflow.

SAY

The corrected contract passes. The separate live workflow goes further: it authenticates to Azure, deploys a candidate, evaluates the exact deployed version, and writes the demo release record only after the required cases pass.

DO

In the successful live workflow, open "Evaluate the exact version and write gated demo release pointer". Show the individual PASS lines and the final 9/9 outcome, then the uploaded evidence artifact.

SAY

Here the evidence is tied to the candidate being evaluated, not just to the fact that a pipeline executed.

An evaluation result and a release policy are different things. The result tells us what happened. The workflow determines what is allowed to happen next. Production would also require the appropriate repository and environment protections; we are not claiming those controls simply appear by default.

DO

Return to the terminal. Show the non-secret release manifest. The live request below has a 30-second on-stage budget. If it takes longer, use the captured registered-batch case and name it as recorded.

COMMAND

```
Get-Content evidence\release.json
```

SAY

This record identifies the agent version and the hashes of the evaluation evidence and datasets. The successful workflow also records the deployment's source revision. Those references let us reopen the evidence behind the decision.

For this next request I will not pass a version on the command line. The demonstration client will read the approved version from this manifest.

COMMAND

```
python scripts\demo.py ask "Check Volt Charge 65 batch CH-0926 for a recall." --out
"$sessionDir\released-request.json"
```

EXPECT

requested_version matches evidence\release.json (version 4 in this package); decision no_recall for registered synthetic batch CH-0926. This is not a blanket safety guarantee. The CLI's no-version path must use the release manifest.

COMMAND

```
$released = Get-Content "$sessionDir\released-request.json" -Raw | ConvertFrom-Json
$released | Select-Object requested_version, variant, hosting
$released.output | Format-List
```

SAY

The client used the approved version. For this registered batch, the fictional register lists no recall. That is a narrower statement than saying the product is safe in every circumstance.

We can now connect the request back to a version and the evidence that supported using it. Again, this is the demo client's version pointer, not a claim that we changed the portal's default endpoint routing.

IF SLOW - SAY

I will use the saved workflow log and release record. These are captured from the real run, not a simulated pipeline, and I will keep the recorded and live parts of the demonstration separate.

DO

Use evidence\github-blocked-contract.txt, github-passing-contract.txt and github-live-release.txt. If the request fails, do not claim a live response. Open the captured candidate row registered-batch instead.

RETURN TO SLIDES - SAY

The release gives us confidence about the cases we tested. It cannot tell us that we anticipated every future customer question.

DO

Return to PowerPoint and advance to slide 12 by minute 34.

Slide 12 - Operations teaches development what to test

TIME

34:00-35:00. Set up the return bridge; do not start a feature tour.

DO

Point to Observe, then the feedback arrow returning to the next cycle.

SAY

This is why the lifecycle is a loop rather than a pipeline that ends at deployment.

Operational traffic gives us evidence about what users actually ask and how the agent responds. Evaluation checks the expectations we have already defined. Review helps us decide which new observations should become additional expectations.

The diagram includes the broader improvement capabilities in Foundry. We are not depending on an Insights scan or an optimizer job to finish live today. Our concrete path is simpler: inspect one interaction, decide the correct behaviour, add a test, and check the corrected candidate.

You have already seen the prepared fix. Now I want to show how it becomes part of a repeatable operating process rather than remaining a one-off correction.

TRANSITION - SAY

Let's carry one observed interaction all the way back into the development contract.

DO

Advance to slide 13 and switch to the prepared operational evidence.

Slide 13 - Close the loop with one new test

TIME

35:00-37:00 inspect operational evidence; 37:00-40:00 review expected behaviour; 40:00-42:00 create the case; 42:00-43:00 return to DEV; 43:00-46:30 focused check and release boundary; 46:30-47:00 return.

DO

Open the pre-bookmarked version-4 trace in Foundry or Application Insights. Use the captured trace ID from evidence\missing-batch-improved.json if needed. The corrected runtime correlates invocation and tool spans. The older baseline capture does not reliably place every tool span under the same trace; use its captured tool_calls record when explaining its defect.

SAY

This is the operational view of the corrected version. We can inspect the invocation and its instrumented tool work after the request has completed.

I am using a rehearsed trace so we do not depend on immediate telemetry ingestion. If you use the Foundry Monitor experience, note that it is preview. The custom spans we need are also available in the connected Application Insights resource.

Now I will open the captured baseline interaction to explain why this regression case belongs in the test set. This is a replay of our prepared incident pattern, not a claim that a new production incident has just occurred.

DO

Show the feedback moment in the recorded evidence HTML, or use:

COMMAND

```
$referenceBaseline | Select-Object requested_version, captured_at, invocation_id, trace_id
$referenceBaseline.output | Format-List
$referenceBaseline.tool_calls | ConvertTo-Json -Depth 10
```

SAY

The enquiry did not include a batch. The returned status was no_recall. The output contains some cautious wording, but the tool has still turned a missing fact into a negative lookup.

Before changing anything, we need to define what should have happened. For this synthetic scenario, my expectation is: ask for the batch code before making a batch-specific recall conclusion, and do not infer safety from missing information.

That is the label we want the next candidate to satisfy. We do not copy the agent's old response into the dataset and call it the correct answer.

DO

Pause for the audience to read that expectation. If someone challenges it, acknowledge that this is exactly why domain review is necessary. For timing, continue with the stated synthetic-demo contract and park customer policy differences for Q&A.

SAY

The next step preserves provenance. We want to know which interaction led to this case, what behaviour the reviewer approved, and which version we checked.

I am acting as the reviewer for this demonstration. The command records the reviewer field and an explicit approval flag, but it is not an authenticated enterprise approval system. Production would need that control enforced through the appropriate identity and workflow.

DO

Only run --approve after you have personally reviewed and accepted the expectation. The unique $sessionDir from preflight avoids overwriting an earlier review. If you are not approving it, skip the command and show the labelled rehearsal row instead.

COMMAND

```
python scripts\demo.py feedback --trace evidence\missing-batch-baseline.json --id reviewed-
missing-batch --query "I have a Volt Charge 65. Is it safe?" --expected-decision needs_batch --
expected-behavior "Ask for the batch code before a recall conclusion; never infer safety from
missing evidence." --reviewer "Sridhar Arrabelly - presenter-reviewed synthetic demo" --approve
--out "$sessionDir\reviewed-case.jsonl"
Get-Content "$sessionDir\reviewed-case.jsonl" | ConvertFrom-Json | Format-List
```

EXPECT

One new JSONL case with expected_decision needs_batch, approved true, reviewer and review time, source invocation ID, source trace ID and source content hash. If the destination already exists, the command refuses to overwrite it. Use a new session directory; do not delete the previous review to make the demo work.

SAY

Here is the resulting case. The query and expected behaviour are separate from the source output. The identifiers preserve the connection to the captured interaction.

Foundry also has a trace-to-dataset experience, currently labelled preview in the documentation checked for this deck. Our demonstrated path is this explicit code-first review, so we do not depend on that portal action being available.

Now we cross back into the inner loop.

DO

At approximately minute 42, switch to VS Code. Show the new JSONL case and the corrected branch in recall_notices. Do not edit a live agent configuration. Give the focused evaluation up to 45 seconds on stage. If it is still running, use the captured reviewed-result fallback rather than delaying the close.

SAY

The operational observation has become a development input. The correction is still made and checked in development; it is not an unreviewed patch to the serving prompt.

Let's run this focused case against the corrected hosted candidate.

COMMAND

```
python scripts\demo.py evaluate --version $improved --dataset "$sessionDir\reviewed-case.jsonl"
--out "$sessionDir\reviewed-result.json"
```

WHILE WAITING - SAY IF NEEDED

The important improvement is not only the response to this one question. We have changed what future candidates must prove. A later modification that reintroduces this behaviour should now be visible to the evaluation process.

EXPECT

PASS reviewed-missing-batch and a 1/1 result. The generic CLI prints "Release gate: PASS" for this focused run, but it is NOT sufficient to approve a full release. scripts\release.py separately requires the complete core and regression case set. Explain this immediately.

SAY

The focused case passes. The CLI calls its result a gate, but this one-case run is not our full release approval. The release script separately requires the complete core and regression suite.

We still have to protect the shopping, compatibility, known-recall and other behaviours we already depended on. Here is the completed full-suite report showing those cases alongside the regression cases.

COMMAND

```
$candidateReport.cases | Select-Object id, passed | Format-Table -AutoSize
$candidateReport | Select-Object requested_version, passed, total, gate_passed
```

SAY

That closes both bridges. An observed interaction became a reviewed test. A corrected candidate was checked against the wider contract. A release decision can now refer to evidence rather than relying on somebody remembering the incident.

IF SLOW - SAY

I will finish with the captured reviewed case and completed result. The learning step is the same: observation, expected behaviour, test, correction and evidence. I am showing a rehearsal record, not claiming a fresh run completed.

DO

Use evidence\reviewed-case.jsonl and evidence\reviewed-result.json for that fallback. The saved focused result predates the final version-4 release; name the version shown in the file rather than describing it as the latest run. Use candidate.json for the completed version-4 full-suite result.

IF DIFFERENT RESULT - SAY

This focused check has not passed. The review still gives us a useful new test, but it does not give us permission to release this result. We would investigate the evidence and keep the current approved version until the acceptance criteria are met.

RETURN TO SLIDES - SAY

We have completed the journey. Let me leave you with the four decisions that make this useful beyond the demonstration.

DO

Return to PowerPoint and advance to slide 14 by minute 47.

Slide 14 - One system. Four things to remember.

TIME

47:00-49:00. Keep 49:00-50:00 as buffer, not additional content.

DO

Slow down. Point to each takeaway once. Do not reopen a demo window.

SAY

First, start with a customer outcome. The charger question gave us a concrete reason for the tools, the data and the acceptance criteria. The architecture served the task rather than becoming the task.

Second, prove the path, not just the prose. We looked at what the tool returned, how the agent used it and which cases passed. A convincing sentence is not enough evidence by itself.

Third, make the release decision explicit. Foundry gives us important runtime and evaluation capabilities, but we have to connect the evidence to a policy that controls what is allowed to be used.

Fourth, turn observations into reviewed tests. That is how an operational surprise becomes a stronger development contract instead of a problem we fix once and later forget.

For a customer pilot, I would begin with one useful journey, agree the failure boundaries and identify the people who own acceptance and operation. Then I would measure the business outcome as well as the technical behaviour.

That is the story I wanted to demonstrate: build with GitHub, operate with Foundry, and connect the lifecycle with evidence.

TRANSITION - SAY

Let's use the remaining time on how this would fit your environment and which customer journey would be worth proving first.

DO

Advance to slide 15. If it is earlier than minute 50, begin Q&A now rather than adding an unplanned feature tour. If behind, stop the recap here.

Slide 15 - Thank you / Q&A

TIME

50:00-60:00. The appendix is optional. Do not read all answers in sequence.

SAY

Thank you. We have looked at one small assistant, but the operating questions apply much more broadly.

Which customer or employee journey would you want to put through this loop? What outcome would make it worthwhile, and what would you need to see before you trusted the next release?

DO

Pause and take questions. Give a direct answer, then ask whether it resolves the question. Use the following routes only when helpful.

IF ASKED "WHY COPILOT SDK RATHER THAN A PROMPT AGENT?" - SAY

A prompt agent may be the simpler choice when its managed authoring and tool capabilities meet the requirement. We chose the Copilot SDK hosted route to show code-owned business logic and the GitHub-to-Foundry release lifecycle. This is not a claim that every agent needs custom hosted code.

IF ASKED "IS THIS THE MANAGED COPILOT-HARNESS PROMPT AGENT?" - SAY

No. This application uses the Copilot SDK in code and is deployed as a hosted agent. That is different from the managed Copilot-harness prompt-agent option.

IF ASKED "CAN IT RUN ARBITRARY CODE OR MODIFY CUSTOMER SYSTEMS?" - SAY

Not in this demonstration. We expose three allowlisted read-only functions. Adding a write action would require its own authorization, validation and approval design; the fact that a model can request an action is not permission to perform it.

IF ASKED "DO NINE PASSING CASES MEAN IT IS SAFE?" - SAY

No. They provide evidence for those cases. A production decision needs broader coverage, calibrated semantic and safety evaluation where appropriate, least privilege, domain review and operational controls.

IF ASKED ABOUT COST OR ROI - SAY

The cost includes inference, hosted compute, telemetry, evaluation and any connected services. I would measure a representative workload and use current rates rather than infer the total from one request's tokens.

For ROI, we would establish a business baseline such as handling time, successful resolution and escalation quality, then measure the pilot against that baseline. This demo does not establish a

financial return.

IF ASKED ABOUT RESIDENCY - SAY

The demo resource is in Sweden Central, but its model deployment is GlobalStandard. That is not a Sweden-only processing commitment. We would validate deployment type, data paths, telemetry, tools and current service terms against your specific requirement.

IF ASKED ABOUT PRODUCTION READINESS - SAY

Use this as a working lifecycle demonstration, not a complete production reference deployment. The next design step is to add the environment separation, identities, mandatory checks, approvals, privacy controls and operating ownership required by your workload.

DO

For implementation boundaries use slide 16; status use 17; broader architecture and ownership use 18; contingency use 19; sources use 20. Return to slide 15 after the answer.

CLOSING - SAY

The most useful next step is a bounded pilot with an agreed evidence contract: one outcome, clear failure boundaries, and a named owner for the release decision. Thank you for the discussion.

Slide 16 - What is implemented—and what is not

TIME

Q&A only, approximately 60 seconds. Return to slide 15 after answering.

DO

Open this slide only for a scope or production-readiness question.

SAY

Let me make the implementation boundary precise.

On the left is what we ran: the Copilot SDK with a Foundry model, a hosted Invocations endpoint, three read-only domain tools, application traces and the evaluation and release-evidence workflow.

On the right is what we have not claimed. A managed Foundry Toolbox and API Management are not configured here. We have not demonstrated a separately isolated customer production environment or an autonomous self-correcting system. And the small test set is not safety certification or measured ROI.

Those are not details to hide. They are the decisions we would make explicitly when taking this pattern into a customer workload.

TRANSITION - SAY

For your scenario, I would identify which of those additional controls are required before choosing the production design.

DO

Return to slide 15 and the question that prompted this appendix.

Slide 17 - Availability is capability-specific

TIME

Q&A only, approximately 60-75 seconds. Status was checked for the deck on 8 November 2026; confirm again before making a customer commitment.

SAY

Foundry's general-availability milestone does not mean every feature, evaluator, tool and portal experience has the same readiness status.

The core hosted-agent, tracing and evaluation capabilities shown here have their documented GA scope. Individual evaluators and tools can differ. Monitoring, Insights, trace-to-dataset conversion and agent-level guardrail experiences have preview boundaries. Agent Optimizer is listed as limited preview in the source checked for this deck.

That is why I have been naming preview experiences when they appear and have not made a live optimizer or Insights run a dependency for the demonstration.

For a real implementation, we would check the exact capability, region, networking path and organizational preview policy. We would not infer a production commitment from a broad platform label.

TRANSITION - SAY

Let's tie the availability question to the specific capability your workload needs.

DO

Return to slide 15. If current status cannot be verified in the session, say: "I will not make an availability commitment from an old screenshot; we should confirm the current feature documentation for that region."

Slide 18 - The broader agent platform

TIME

Q&A only, approximately 75-90 seconds. Do not enumerate every product in the picture.

DO

Point to Build in GitHub, Run in Foundry, and the governance/context layers.

SAY

This wider platform picture helps place the narrow demonstration in context.

GitHub is where the application can be developed and changed like software. Foundry provides the model and runtime environment, together with lifecycle capabilities for evaluation and operation. Context, integration, governance and distribution are additional design layers around that application.

The important question is not only which product sits in each box. It is who owns each responsibility.

The application team owns the business contract, tool correctness and user experience. Platform and security teams help define identity, permissions, networking and telemetry controls. The business and operating owners determine what acceptance means and what happens when the service behaves unexpectedly.

Managed hosting reduces infrastructure work; it does not make the application's decisions correct automatically. That is why the evidence bridges remain important regardless of the framework or channel.

This is the broader reference picture. The demo has not configured every integration shown here.

TRANSITION - SAY

I would use this diagram to agree ownership and boundaries for your pilot, then keep the implementation focused on the first measurable outcome.

DO

Return to slide 15.

Slide 19 - When a live step takes too long

TIME

Presenter contingency reference. Normally remain hidden.

DO

Use this policy whenever a live segment overruns; you do not need to show the slide to the customer. At 43:00, move to the final regression and closing story. At 50:00, stop adding demonstration content and open Q&A.

If five minutes behind, keep one live request, use completed scorecards and workflow records, and preserve the observation-to-reviewed-test connection. Skip fresh batch evaluations, cold deployment and optional preview screens.

IF SWITCHING TO RECORDED EVIDENCE - SAY

Rather than spend your time waiting, I am switching to a captured rehearsal result from this same project. I will show the version and timestamp so it is clear what is recorded and what we ran live.

IF THE CLOUD IS UNAVAILABLE - SAY

The cloud step is not available in this session. I can still walk you through the recorded execution and the code contract, but I will not describe an offline test as a live model result.

IF AN EXPECTED ASSERTION FAILS - SAY

The expected behaviour did not pass. In the operating process we have just discussed, that means retain the evidence and investigate; it does not mean change the expectation to make the demonstration green.

IF A PORTAL CONTROL IS MISSING - SAY

That portal path is not available in this environment. I will use the code-first path we have rehearsed and keep the capability boundary explicit.

TRANSITION - SAY

The point to retain is the evidence trail: an observation becomes a reviewed expectation, a test, and a candidate that must prove the change.

DO

Return to the next planned slide, or slide 14 if the main session is at minute 47.

Slide 20 - Sources and useful boundaries

TIME

Q&A only, approximately 45-60 seconds.

SAY

There are three kinds of source behind this session.

The workshop supplied the Volt & Co use case and the lifecycle diagrams. The Bank Briefing supplied the visual style and original Azure artwork. Those are reference materials, not evidence that the current demo executed every capability pictured.

The concrete result claims come from this project's captured runs: the identified agent versions, the synthetic datasets, the native evaluation records, the traces and the GitHub workflow.

Product and availability claims are grounded in the official documentation linked in the notes and repository. No NDA roadmap pages are included, and no other agent's result is presented as a Volt & Co result.

TRANSITION - SAY

If there is a specific implementation or availability point you want to inspect, we can anchor the discussion to that source rather than make a broader claim.

DO

Return to slide 15.

REFERENCE - NOT READ ALOUD

https://learn.microsoft.com/azure/foundry/agents/concepts/development-lifecycle

https://learn.microsoft.com/azure/foundry/agents/concepts/hosted-agents

https://learn.microsoft.com/azure/foundry/observability/how-to/cloud-evaluation-datasets

https://learn.microsoft.com/azure/foundry/observability/how-to/trace-agent-setup

https://learn.microsoft.com/azure/foundry/observability/how-to/how-to-monitor-agents-dashboard

https://learn.microsoft.com/azure/foundry/concepts/general-availability

https://learn.microsoft.com/azure/foundry/agents/quickstarts/set-up-cicd-hosted-agent

https://github.com/github/copilot-sdk/blob/main/docs/auth/byok.md